

Содержание

1	Понятие обучения с подкреплением	2
1.1	Базовые определения и постановка задачи	2
1.2	Компромисс изучение-применение	5
1.3	Постановка задачи о k -руком бандите	6
1.4	Жадная стратегия	10
1.5	ε -жадная стратегия	13
1.6	Softmax-стратегия	17
1.7	Метод UCB (Upper Confidence Bound)	19
1.8	Оптимистичные начальные оценки	22
2	Общая постановка задачи	24
2.1	Взаимосвязь агента и окружающей среды	24
2.2	Функции ценности действий и состояний	32
2.3	Пример	35
2.4	Оценка стратегий	39
2.5	Оптимальные стратегии	46
2.6	SARSA	55
2.7	Q-обучение	60
2.8	Пример	63
2.9	Резюме	65
3	Заключение	65

2.6 SARSA

Рассмотренные ранее методы обучения имеют достаточно серьезный недостаток. Дело в том, что они предполагают тот факт, что агент имеет представление о конфигурации среды. Иными словами, в уравнении Беллмана

$$v_{\pi}(s) = \sum_{a \in A(s)} \pi(a|s) \sum_{s' \in S} \mathcal{P}_{ss'}^a (\mathcal{R}_{ss'}^a + \gamma v_{\pi}(s')),$$

предполагаются известными и $\mathcal{P}_{ss'}^a$, то есть вероятности перехода из состояния s в состояние s' при совершении действия a , и все ожидаемые премии, то есть $\mathcal{R}_{ss'}^a$. В реальности это почти всегда не так. Очевидный выход – производить некоторые оценки на основе выполненных действий и полученных результатов. Идея очень похожа на идею, которая ранее была рассмотрена в разделе про многоруких бандитов. Вспомним рекуррентную формулу для нахождения ценности некоторого действия на основе экспоненциального скользящего среднего.

$$q_{t+1}(a) = q_t(a) + \alpha (r_{t+1} - q_t(a)),$$

где $\alpha \in (0, 1]$. Такой подход применим и для оценки ценности состояний. Он получил название Temporal Difference (TD). Учитывая, что

$$v_{\pi}(s) = \mathbb{E}_{\pi}(G_t | s_t = s) = \mathbb{E}_{\pi} \left(\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | s_t = s \right) = \mathbb{E}_{\pi} (r_{t+1} + \gamma v_{\pi}(s') | s_t = s),$$

можно получить способ оценки ценности состояния s_t :

$$v(s_t) \leftarrow v(s_t) + \alpha (r_{t+1} + \gamma v(s_{t+1}) - v(s_t))$$

В данном случае, находясь в состоянии s_t агент, следуя какой-то стратегии, выполняет действие a_t , получает награду r_{t+1} и переходит в состояние s_{t+1} .

Ценность $v(s_{t+1})$ нового состояния s_{t+1} также известна, а значит можно обновить ценность состояния s_t . Иными словами, на следующем шаге обновляется значение оценки ценности для предыдущего шага.

Оценка ценности состояний хоть и важна, но не является самоцелью, так как награду приносят действия, а они выбираются в зависимости от стратегии. Поэтому при обучении нужно оптимизировать и стратегию. Для этих целей можно использовать идею TD для оценки функции действий $q(s, a)$, а именно

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha(r_{t+1} + \gamma q(s_{t+1}, a_{t+1}) - q(s_t, a_t))$$

Можно также заметить, что в процессе обучения повторяется одна и та же последовательность: состояние, действие, награда, состояние,..., то есть

$$s, a, r, s, a, r, s, a, r, \dots,$$

поэтому такой метод и получил название *SARSA*. Зная оценку ценности каждого действия, можно использовать, например, жадную стратегию для выбора очередного действия, то есть стратегию, задаваемую следующим образом:

$$A_t = \underset{a \in A(s_t)}{\text{Arg max}} q_t(a), \quad \pi(a|s_t) = \begin{cases} \frac{1}{|A_t|}, & a \in A_t \\ 0, & \text{иначе} \end{cases},$$

но все же логично уделять некоторое внимание и исследованию. Для таких целей подходит, например, ранее рассмотренная ε -жадная стратегия:

$$\pi_t(a|s_t) = \begin{cases} \frac{1 - \varepsilon}{|A_t|} + \frac{\varepsilon}{|A(s_t)|}, & a \in A_t \\ \frac{\varepsilon}{|A(s_t)|}, & a \notin A_t \end{cases},$$

Ниже приведено формальное описание алгоритма *SARSA*.

Алгоритм 3. Алгоритм SARSA

-
- 1: Пусть S – множество состояний, $A(s)$, $s \in S$ – множество доступных действий в состоянии s .
 - 2: Инициализировать $q(s, a)$, $s \in S$, s – нетерминальное, $a \in A(s)$ произвольно
 - 3: Инициализировать α и γ
 - 4: **Для** каждой игры повторять **Выполнять**
 - 5: Инициализировать случайным образом нетерминальное состояние s_0
 - 6: Выбрать a_0 согласно стратегии $\pi_0(a|s_0)$
 - 7: $t \leftarrow 0$
 - 8: **Для** каждого шага игры t , пока не выполнен критерий остановки или пока s_t – не терминальное состояние, **Выполнять**
 - 9: Выполнить a_t , найти r_{t+1} , перейти в s_{t+1}
 - 10: **Если** s_{t+1} – терминальное, **тогда**
 - 11: $q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha(r_{t+1} - q(s_t, a_t))$
 - 12: **иначе**
 - 13: Выбрать a_{t+1} на основе стратегии $\pi_{t+1}(a|s_{t+1})$
 - 14: $q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha(r_{t+1} + \gamma q(s_{t+1}, a_{t+1}) - q(s_t, a_t))$
 - 15: **Конец условия**
 - 16: $t \leftarrow (t + 1)$
 - 17: **Конец цикла**
 - 18: **Конец цикла**
-

Итак, согласно алгоритму, на первом этапе произвольно инициализируются ценности всех доступных действий $a \in A(s)$ для каждого нетерминального состояния $s \in S$. Далее проводится серия игр. В каждой игре начальное нетерминальное состояние агента инициализируется случайным образом. Далее, в рамках каждой игры на каждом шаге t агент, находясь в состоянии s_t , выбирает некоторое действие a_t согласно выбранной стратегии, переходит в следующее состояние s_{t+1} и получает вознаграждение r_{t+1} . Если состояние s_{t+1} – не терминальное, то согласно стратегии выбирается действие a_{t+1} , а значение $q(s_t, a_t)$ обновляется согласно формуле.

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha(r_{t+1} + \gamma q(s_{t+1}, a_{t+1}) - q(s_t, a_t)).$$

Далее происходит переход на новую итерацию игры. Если состояние s_{t+1} является терминальным, то $A(s_{t+1}) = \emptyset$ и

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha(r_{t+1} - q(s_t, a_t)).$$

Игра завершается в случае, если совершен переход в терминальное состояние или выполнен критерий остановки. Примерами выполнения критерия остановки могут служить следующие ситуации:

- достигнут заданный уровень суммы полученных наград;
- достигнут заданный уровень среднего значения полученных наград;
- достигнут заданный уровень числа потраченных шагов;
- и тому подобное.

Рассмотрим применение алгоритма *SARSA* для уже знакомого, хотя и слегка модифицированного примера: будем считать состояние «Сон» терминальным (при переходе в него игра заканчивается). Схема представлена на рисунке [19](#).



Рис. 19: Описание среды

Положим $\gamma = 0.8$, $\alpha = 0.1$. В качестве стратегии будем использовать ε -жадную стратегию со значением $\varepsilon = 0.1$. Отметим также, что теперь нам неизвестны ни вероятности переходов из состояния s в состояние s' в результате выполнения действия a , ни то, какие награды мы получим в этом случае. Все значения придется оценивать по ходу игры. Для удобства сформируем

таблицу ценностей действий в зависимости от состояний и произведем начальную инициализацию (установим все возможные значения ценностей равными нулю). Обращаем внимание, что символ $\times$ означает, что действие не может быть выбрано в текущем состоянии.

	a_1	a_2	a_3	a_4
s_1	0	0	$\times$	$\times$
s_2	0	$\times$	$\times$	0
s_3	$\times$	0	0	$\times$
s_4	$\times$	0	$\times$	0
s_5	$\times$	$\times$	$\times$	$\times$

Предположим, что мы начинаем в состоянии s_1 . В нем нам доступны действия a_1 и a_2 , причем на данный момент $q(s_1, a_1) = q(s_1, a_2) = 0$. Согласно нашей стратегии, в данном случае произвольным образом выбирается любое из доступных действий. Пусть выбрано действие a_2 . Тогда агент переходит в состояние s_2 и получает награду $r = -2$. Далее, находясь в состоянии s_2 , нужно выбрать следующее действие. Нам доступны a_1 и a_4 . Так как ценности одинаковые (равны нулю), выбираем любое, например a_4 . Теперь можно обновить $q(s_1, a_2)$.

$$\begin{aligned} q(s_1, a_2) &\leftarrow q(s_1, a_2) + \alpha(r + \gamma q(s_2, a_4) - q(s_1, a_2)) = \\ &= 0 + 0.1 \cdot (-2 + 0.8 \cdot 0 - 0) = -0.2. \end{aligned}$$

Занесем полученное значение в таблицу.

	a_1	a_2	a_3	a_4
s_1	0	-0.2	$\times$	$\times$
s_2	0	$\times$	$\times$	0
s_3	$\times$	0	0	$\times$
s_4	$\times$	0	$\times$	0
s_5	$\times$	$\times$	$\times$	$\times$

Продолжим выполнение алгоритма.

Текущее состояние: s_2

Выбранное действие: a_4

Следующее состояние: s_5

Награда: 7

Отметим, что так как s_5 – терминальное состояние, то игра заканчивается.

Тогда

$$q(s_2, a_4) \leftarrow 0 + 0.1 \cdot (7 - 0) = 0.7.$$

	a_1	a_2	a_3	a_4
s_1	0	-0.2	×	×
s_2	0	×	×	0.7
s_3	×	0	0	×
s_4	×	0	×	0
s_5	×	×	×	×

Произведя еще несколько игр, получим следующую таблицу.

	a_1	a_2	a_3	a_4
s_1	-0.32	0.88	×	×
s_2	-0.02	×	×	6.5
s_3	×	-0.09	-0.19	×
s_4	×	0.99	×	6.14
s_5	×	×	×	×

Итак, для оценки выполненного действия мы регулярно как бы заглядываем в будущее и заранее выбираем следующее действие, что, в конечном счете, сказывается на оценке текущего действия.

Замечание 2.6.1 Заметим, что на основе последней таблицы можно сформировать оптимальную стратегию (если выбирать действия жадно), которая немного отличается от найденной теоретической из-за терминальности состояния s_5 (рисунок 20). Находясь в состоянии s_1 , надо выбрать действие a_2 , так как

$$q(s_1, a_2) = 0.88 > q(s_1, a_1) = -0.32.$$

Находясь в состоянии s_2 – выбирать действие a_4 , так как

$$q(s_2, a_4) = 6.5 > q(s_2, a_1) = -0.02$$

и так далее.

Замечание 2.6.2 В нашем примере получаемые в процессе игры награды имеют одни и те же значения вне зависимости от s , s' и a . В более общем случае это может быть не так.

2.7 Q-обучение

Рассмотрим еще один алгоритм обучения на основе TD. Ранее нами были получены уравнения оптимальности Беллмана, в частности уравнение для оптимальной функции ценности действий

$$q^*(s, a) = \sum_{s' \in S} \mathcal{P}_{ss'}^a \left(\mathcal{R}_{ss'}^a + \gamma \max_{a' \in A(s')} q^*(s', a') \right).$$



Рис. 20: Оптимальная стратегия

Идея метода Q -обучения заключается в аппроксимации оптимальной функции ценности следующим образом

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \left(r_{t+1} + \gamma \max_{a \in A(s_{t+1})} q(s_{t+1}, a) - q(s_t, a_t) \right).$$

Стоит на секунду задуматься: а в чем отличие $SARSA$ и Q -обучения? Дело в том, что для обновления $q(s_t, a_t)$ в $SARSA$ используется значение $q(s_{t+1}, a_{t+1})$, то есть ценность действия a_{t+1} , выбранного в состоянии s_{t+1} согласно некоторой установленной стратегии $\pi_{t+1}(a|s_{t+1})$. В Q -обучении используется оптимальная стратегия, так как выбирается то действие, для которого достигается $\max_{a \in A(s_{t+1})} q(s_{t+1}, a)$. В итоге при обновлении значения функции ценности действия мы опираемся не на выбранную стратегию (возможно, не оптимальную), а на оптимальную, но на следующем шаге (!). Если текущая стратегия оптимальна, то алгоритмы работают одинаково. Запишем

алгоритм Q -обучения формально.

Алгоритм 4. Алгоритм Q -обучения

- 1: Пусть S – множество состояний, $A(s)$, $s \in S$ – множество доступных действий в состоянии s .
 - 2: Инициализировать $q(s, a)$, $s \in S$, s – нетерминальное, $a \in A(s)$ произвольно
 - 3: Инициализировать α и γ
 - 4: **Для** каждой игры повторять **Выполнять**
 - 5: Инициализировать случайным образом нетерминальное состояние s_0
 - 6: $t \leftarrow 0$
 - 7: **Для** каждого шага игры t , пока не выполнен критерий остановки или пока s_t – не терминальное состояние, **Выполнять**
 - 8: Выбрать a_t согласно стратегии $\pi_t(a|s_t)$
 - 9: Выполнить a_t , найти r_{t+1} , перейти в s_{t+1}
 - 10: **Если** s_{t+1} – терминальное, **тогда**
 - 11: $q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha(r_{t+1} - q(s_t, a_t))$
 - 12: **иначе**
 - 13: $q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha(r_{t+1} + \gamma \max_{a \in A(s_{t+1})} q(s_{t+1}, a) - q(s_t, a_t))$
 - 14: **Конец условия**
 - 15: $t \leftarrow (t + 1)$
 - 16: **Конец цикла**
 - 17: **Конец цикла**
-

Во-первых, как мы уже отмечали ранее, разница с *SARSA* заключается в способе оценки ценности действия. Во-вторых отличается выбор действия: в *SARSA* на текущей итерации находилось как текущее так и следующее действие, в то время как в Q -обучении происходит выбор только текущего действия. Если состояние s_{t+1} – не терминальное, то $q(s_t, a_t)$ обновляется согласно формуле

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha(r_{t+1} + \gamma \max_{a \in A(s_{t+1})} q(s_{t+1}, a) - q(s_t, a_t)).$$

Далее происходит переход на новую итерацию игры. Если состояние s_{t+1} является терминальным, то $A(s_{t+1}) = \emptyset$ и

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha(r_{t+1} - q(s_t, a_t)).$$

Игра завершается в случае, если выполнен критерий остановки, или совершен переход в терминальное состояние.

Поясним отличия алгоритма Q -обучения и алгоритма *SARSA* на все том же примере. Пусть на каком-то этапе обучения агент окажется в состоянии s_1 , выберет действие a_2 , а таблица оценок будет иметь вид

	a_1	a_2	a_3	a_4
s_1	-0.32	0.88	×	×
s_2	-0.02	×	×	6.5
s_3	×	-0.09	-0.19	×
s_4	×	0.99	×	6.14
s_5	×	×	×	×

Тогда уточнение оценки $q(s_1, a_2)$ вне зависимости от используемой стратегии будет произведено следующим образом:

$$q(s_1, a_2) \leftarrow 0.88 + 0.1 \cdot (-2 + 0.8 \cdot 6.5 - 0.88) \approx 1.12,$$

так как

$$q(s_2, a_4) = \max(q(s_2, a_1), q(s_2, a_4)).$$

В алгоритме *SARSA* это не всегда так. Например, если бы в результате использования ε -жадной стратегии в состоянии s_2 было выбрано действие не a_4 , а a_1 , то для обновления ценности $q(s_1, a_2)$ использовалось бы значение $q(s_2, a_1)$. В случае *Q*-обучения стратегия (например ε -жадная) используется только при выборе текущего действия и не участвует в обновлении оценки ценности.

2.8 Пример

Рассмотрим примеры использования рассмотренных алгоритмов. Симулируем игру, в которой агенту нужно найти выход из лабиринта, пример возможного решения представлен на рисунке [21](#).

В процессе игры агент помещается в точку старта, ему доступны перемещения в четырех направлениях (вверх, вправо, вниз и влево). Цель агента – достичь точки выхода из лабиринта. В результате каждого действия агент получает награду. Награды суммируются в процессе игры. Каждый шаг сопровождается наградой -0.05 . Если агент переместился в уже посещенную ячейку, награда составляет -0.25 . Если в результате действия агент упирается в стену, награда составляет -0.75 . Выход из лабиринта сопровождается наградой $+10$. В рамках одной игры терминальное состояние достигается всегда: либо игрок находит выход и тогда игра завершается победой, либо сумма полученных наград окажется меньше заданного порогового значения. В описываемом случае пороговое значение – это

$$-\frac{1}{2} \cdot (\text{Размер лабиринта}) = -32.$$

Для обучения использовался ε -жадный алгоритм со значением $\varepsilon = 0.1$. Коэффициент дисконтирования γ равен 0.9, коэффициент $\alpha = 0.1$.

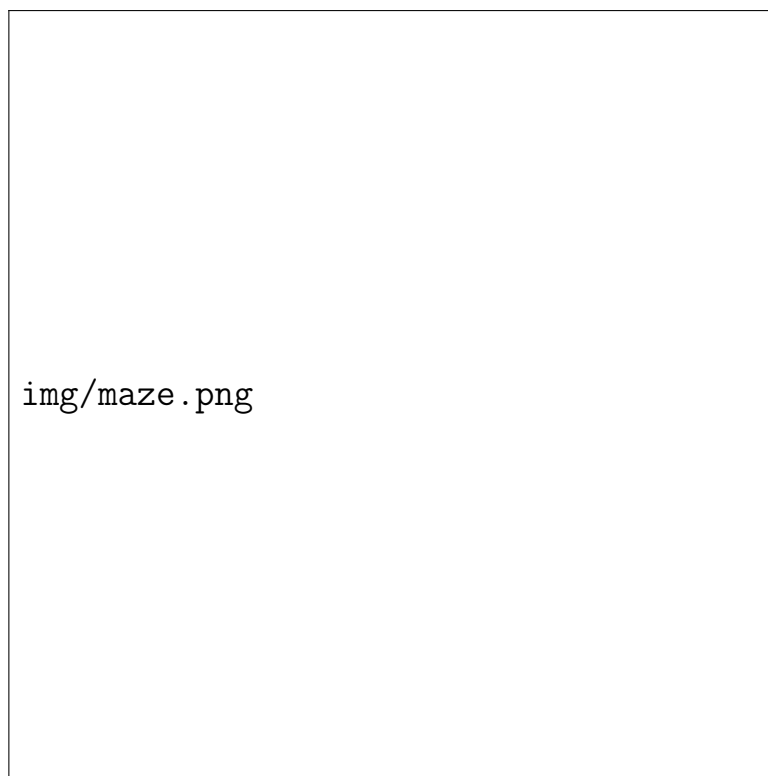


Рис. 21: Пример выхода из лабиринта.

Агенту доступно 44 начальных состояния (белые клетки за исключением клетки выхода из лабиринта). Эпизодом будем называть серию из 44 игр, где каждое состояние (клетка) используется ровно один раз в качестве начального. Обучение заканчивается, если агент умеет находить путь к выходу из любого из 44 доступных состояний (не обязательно оптимальный). Иными словами, оказавшись в любом состоянии, агент точно знает какие действия нужно совершать, чтобы гарантированно выйти из лабиринта.

На основе полученных значений ценностей действий для всех состояний можно составить оптимальную стратегию. Полученные стратегии для алгоритмов *SARSA* и *Q*-обучения представлены на рисунках 22а) и 22б), соответственно.

Можно заметить некоторые интересные различия, связанные с использованием ϵ -жадных стратегий. Посмотрите на состояние (3, 3) (клетка, отвечающая пересечению третьей строки и третьего столбца). Если начинать в этом состоянии, то стратегия, полученная согласно методу *SARSA*, предлагает следовать вниз, и тогда выход будет найден за 17 шагов. Стратегия, полученная на основе *Q*-обучения, наоборот говорит идти вверх, но тогда выход будет найден через 21 шаг. Для этого состояния *SARSA* оказался более эффективным. С другой стороны, у *SARSA* можно заметить некоторую странность в ячейке (6, 3) (пересечение 6 строки и 3 столбца). Различия, например, как в ячейке (5, 8), роли не играют.

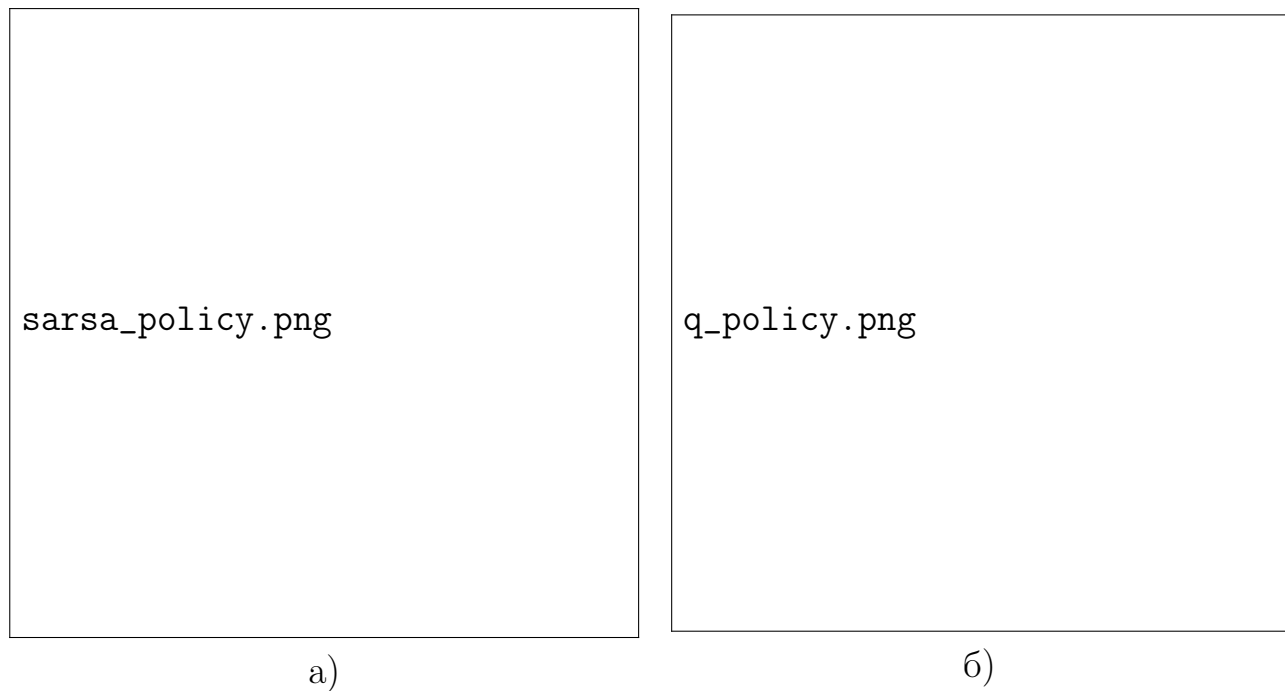


Рис. 22: Стратегии алгоритмов *SARSA* и *Q*-обучения.

2.9 Резюме

В заключение отметим, что обучение с подкреплением – это, возможно, одно из самых интересных и многообещающих направлений машинного обучения. Такой тип обучения сочетает в себе как обучение с учителем, так и обучение без учителя. С одной стороны, человеческого вмешательства не требуется, агент обучается самостоятельно. С другой стороны, учителем в некотором смысле выступает среда. Еще одним существенным плюсом является то, что обучение можно производить в динамично меняющейся среде. Отметим, что в этой лекции мы рассмотрели лишь небольшую часть методов обучения с подкреплением, которых на самом деле достаточно много. При этом новые решения и идеи в этой области публикуются с завидной регулярностью.